

SLURRY_FREE_ACID

Macro and Micro Integration Architecture

Advisory industrial ML deployment - CSV replay, OPC UA, Docker and SQLite

Prepared by	Ikram Benfella
Current stage	Stage 1 laboratory simulation
Safety position	Read-only/advisory; no PLC or DCS control writes
Validated output	SQLite prediction audit plus JSON mirror

1. Macro architecture: plant to operator

Decision: the model is an isolated advisory layer. Plant control and safety remain in the PLC/DCS even when ML, Docker, OPC or storage fails.

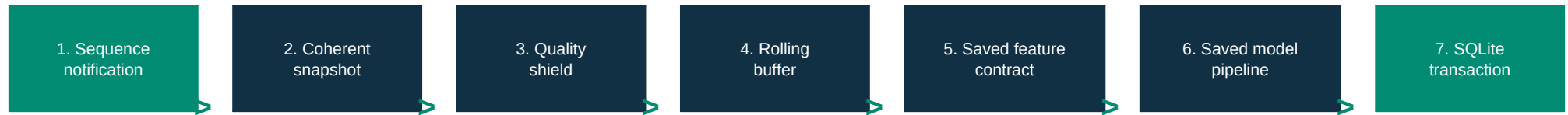


Layer	Component	Responsibility	If it fails
Physical	Sensors / laboratory	Measure process and free-acid samples	Controller applies existing safety behavior
Control	PLC / DCS	Deterministic control, alarms, interlocks	Independent of the ML service
Translation	Kepware	Vendor drivers southbound; OPC UA northbound	Inference disconnects and retries
Compute	Docker inference	Validate, buffer, engineer features, predict	No control function is lost
Persistence	SQLite + JSON	Audit runs, results, quality and status	No unaudited advisory result is released
Presentation	HMI / dashboard	Show estimate, timestamp, quality and status	Operator retains authority

Interfaces

- Field to controller: 4-20 mA or digital I/O; existing plant responsibility.
- Controller to Kepware: vendor-specific industrial protocol, to be confirmed with IT/OT.
- Kepware to Python: OPC UA subscription with timestamps and quality codes.
- Python to user: persisted advisory result; optional dedicated HMI tags only after approval.

2. Micro architecture: one inference cycle



Step	Implementation	Reason / output
1	Subscribe to the OPC sequence tag	Avoid repeated polling; process each committed source row once
2	Read sequence before and after all mapped tags	Reject a mixed snapshot if a new row was committed during reading
3	Check OPC quality, timestamp, cadence, ranges and missing share	Isolated gaps use trained imputation; >25% missing required inputs is rejected
4	Append accepted rows to a bounded chronological buffer	Preserve the exact history needed by temporal features
5	Wait for 31 rows on a cold start, then build saved columns	30-minute lookback plus current row; no invented startup history
6	Apply persisted preprocessing and selected model without refitting	Training/production consistency and leakage prevention
7	Commit SQLite row, mirror JSON, optionally publish advisory tags	Auditable result before presentation; no control tags

Operational modes

Mode	Target required at inference	Meaning
target_free	No	5-cluster Ridge virtual sensor estimates current free acid from process inputs
target_anchored	Fresh manual laboratory value + timestamp	Predicts 1/5/10/15-minute change and adds it to the known current level

Important: the laboratory value is manual. It cannot be silently treated as a continuously available online analyzer value.

3. SQLite output and traceability

Decision: SQLite is the authoritative Stage-1 audit store. It is embedded in the inference process, not a separate network server. JSONL/latest JSON remain readable mirrors.

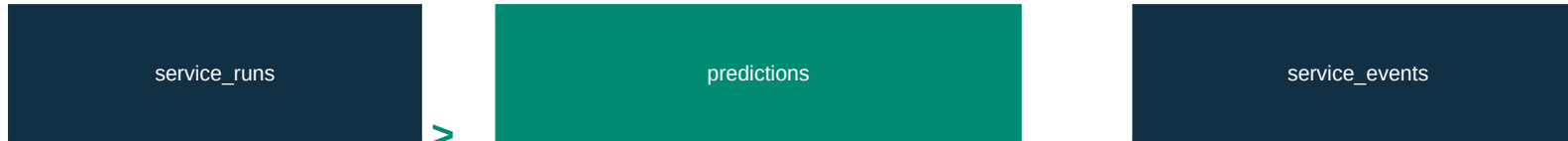


Table	One row represents	Important fields and controls
service_runs	One service startup	run UUID, service/model version, mode, horizon, start time, configuration snapshot
predictions	One successful prediction	source and forecast timestamps, result/change, quality, imputed inputs, model; UNIQUE(run_id, sequence)
service_events	One status transition	STARTING, CONNECTED, WARMING_UP, RUNNING, DATA_ERROR or DISCONNECTED with context

Reliability choices

- Each prediction is inserted transactionally; incomplete database rows are not possible.
- WAL mode supports an inspection reader while the inference process writes.
- A run UUID separates restarts and repeated CSV replays; sequence uniqueness is enforced within a run.
- Timestamp and quality indexes support event review and performance validation.
- The Docker bind mount keeps the database outside the container lifecycle.

Validated Stage-1 result

The rebuilt container passed storage tests and produced live SQLite records from the OPC replay. The inspection command returned model name clustered_ridge_k5, process timestamps, predictions and GOOD input-quality status; CSV export also completed.

4. Integration sequence and decisions still required

Stage	What is connected	Acceptance evidence
1 - completed prototype	CSV replay OPC server -> Docker inference -> SQLite/JSON	Build, subscription, warm-up, prediction, status, persistence and export
2 - Kepware laboratory	Approved Kepware instance -> Docker client	Real Node IDs, certificates, SignAndEncrypt, reconnect and quality-code tests
3 - production shadow	Read-only plant tags -> silent prediction storage	Timestamp alignment, laboratory matching, drift and prospective error review
4 - advisory HMI	Dedicated model output/status tags -> operator display	Human factors, alarm policy, ownership and failure-state approval

Information to obtain from the supervisor / IT-OT team

- DCS and PLC vendors/versions; exact meaning and scope of the plant 'automate'.
- OPC UA versus OPC DA, Kepware endpoint, real Node IDs for all process variables, security policy and certificates.
- Network route and firewall rule between the Windows Docker host and Kepware host.
- Authoritative source timestamps, OPC quality semantics and clock synchronization.
- Manual laboratory sampling/entry workflow and how a sample time differs from a database entry time.
- Database retention, backup location, responsible owner and whether the plant historian becomes the long-term record.
- Approved advisory tags and HMI behavior for warming-up, imputed data, data error and disconnection.

Safety conclusion

The current architecture is suitable for laboratory and read-only shadow validation. It is not approved for automatic intervention. The next technical milestone is a secured Kepware laboratory connection followed by prospective comparison with timestamped laboratory results.